



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



Syntax Analysis –Top Down

Presented by,
Dr. Baiju B V
Assistant Professor
SCOPE, VIT, Vellore

LEFT RECURSION & LEFT FACTORING

LEFT RECURSION

- A grammar is said to be **left recursive** if it has a non terminal A such that there is a derivation $A \rightarrow A\alpha$ for some string α .

Left recursion elimination

$$A \rightarrow A\alpha \mid \beta \quad \longrightarrow \quad \begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \epsilon \end{array}$$

- The production is **left-recursive** if the leftmost symbol on the right side is the same as the non-terminal on the left side.

Examples: Left recursion elimination

1. Eliminate left recursion for the following grammar

$$\begin{array}{l} E \rightarrow E+T \mid T \\ T \rightarrow T*F \mid F \\ T \rightarrow (E) / \text{id} \end{array}$$
$$A \rightarrow A\alpha \mid \beta \longrightarrow \begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \epsilon \end{array}$$

$E \rightarrow E+T \mid T$ (RHS and LHS contain same non terminal symbol)

$$\begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \epsilon \end{array}$$

$T \rightarrow T*F \mid F$

$$\begin{array}{l} T \rightarrow FT' \\ T' \rightarrow *FT' \mid \epsilon \end{array}$$

Grammar after left recursion

$$\begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \epsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid \epsilon \\ T \rightarrow (E) / \text{id} \end{array}$$

Left-Recursion

Problem-1: Consider the following grammar and eliminate left recursion-

$$A \rightarrow AA\alpha / \beta$$

Solution:

The grammar after eliminating left recursion is-

$$A \rightarrow \beta A'$$

$$A' \rightarrow A\alpha A' / \epsilon$$

Problem-2: Consider the following grammar and eliminate left recursion-

$$S \rightarrow S0S1S / 01$$

Solution:

The grammar after eliminating left recursion is-

$$S \rightarrow 01A$$

$$A \rightarrow 0S1SA / \epsilon$$

Left-Recursion

Problem-3: Consider the following grammar and eliminate left recursion-

$A \rightarrow ABd / Aa / a$

$B \rightarrow Be / b$

Solution:

The grammar after eliminating left recursion is-

$A \rightarrow aA'$

$A' \rightarrow BdA' / aA' / \epsilon$

$B \rightarrow bB'$

$B' \rightarrow eB' / \epsilon$

Problem-4: Consider the following grammar and eliminate left recursion-

$E \rightarrow E + E / E \times E / a$

Solution:

The grammar after eliminating left recursion is-

$E \rightarrow aA$

$A \rightarrow +EA / \times EA / \epsilon$

Exercise: Left recursion

1. $A \rightarrow Abd \mid Aa \mid a$

$$B \rightarrow Be \mid b$$

2. $A \rightarrow AB \mid AC \mid a \mid b$

3. $S \rightarrow A \mid B$

$$A \rightarrow ABC \mid Acd \mid a \mid aa$$

$$B \rightarrow Bee \mid b$$

4. $\text{Exp} \rightarrow \text{Exp} + \text{term} \mid \text{Exp} - \text{term} \mid \text{term}$

Ex: 1

$$A \rightarrow Abd \mid Aa \mid a$$

$$B \rightarrow Be \mid b$$

Soln:

$$A \rightarrow a A'$$

$$A' \rightarrow bdA' \mid aA' \mid \epsilon$$

Ex: 2

$$A \rightarrow AB \mid AC \mid a \mid b$$

Soln:

$$A \rightarrow aA' \mid bA'$$

$$A' \rightarrow BA' \mid CA' \mid \epsilon$$

Ex: 3: $S \rightarrow A \mid B$; $A \rightarrow ABC \mid Acd \mid a \mid aa$; $B \rightarrow Bee \mid b$

Soln:

$$S \rightarrow A \mid B$$

$$A \rightarrow aA' \mid aaA'$$

$$A' \rightarrow BCA' \mid cdaA' \mid \epsilon$$

$$B \rightarrow bB'$$

$$B' \rightarrow eeB' \mid \epsilon$$

Ex: 4:

$$\text{Exp} \rightarrow \text{Exp} + \text{term} \mid \text{Exp} - \text{term} \mid \text{term}$$

Soln:

$$\text{Exp} \rightarrow \text{term} A'$$

$$A' \rightarrow + \text{term} A' \mid - \text{term} A' \mid \epsilon$$

LEFT FACTORING

- Left factoring is a **grammar transformation** that is useful for **producing a grammar suitable for predictive parsing**.
- Left factoring is a process by which the *grammar with common prefixes is transformed*.
- The common prefix may be a *terminal* or a *non-terminal* or a *combination of both*.
- Grammar in the following form will be considered to be having left factored

$$S \Rightarrow aX \mid aY \mid aZ$$

- S , X , Y , and Z are non-terminal symbols, and a is terminal.
- There are three different productions with the same non-terminal symbol on the left-hand side, and the productions have a common prefix a .

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \dots \dots \mid r_1 \mid r_2$$



$$\begin{aligned} A &\rightarrow \alpha A' \mid r_1 \mid r_2 \\ A' &\rightarrow \beta_1 \mid \beta_2 \dots \end{aligned}$$

Do left factoring in the following grammar.

$$S \rightarrow \textcolor{blue}{a}AB \mid \textcolor{red}{a}CD$$

$$S \rightarrow aS'$$

$$S' \rightarrow AB \mid CD$$

$$A \rightarrow \textcolor{blue}{x}B\textcolor{red}{y}A \mid \textcolor{red}{x}B\textcolor{blue}{y}A\textcolor{blue}{z}A \mid a$$

$$A \rightarrow xByAA' \mid a$$

$$A' \rightarrow \epsilon \mid zA$$

$$A \rightarrow \textcolor{red}{a}AB \mid \textcolor{red}{a}A \mid \textcolor{red}{a}$$

$$A \rightarrow aA'$$

$$A' \rightarrow \textcolor{red}{A}B \mid \textcolor{red}{A} \mid \epsilon$$

$$A' \rightarrow AA'' \mid \epsilon$$

$$A'' \rightarrow B \mid \epsilon$$

$$A \rightarrow \textcolor{red}{\alpha}\textcolor{blue}{\beta}_1 \mid \textcolor{red}{\alpha}\textcolor{blue}{\beta}_2 \mid \dots \dots \dots \mid \textcolor{blue}{r}_1 \mid \textcolor{blue}{r}_2$$



$$\textcolor{blue}{A} \rightarrow \textcolor{red}{\alpha}\textcolor{blue}{A}' \mid \textcolor{blue}{r}_1 \mid \textcolor{blue}{r}_2$$

$$\textcolor{blue}{A}' \rightarrow \textcolor{blue}{\beta}_1 \mid \textcolor{blue}{\beta}_2 \dots$$

Exercise

1. $S \rightarrow iEtS \mid iEtSeS \mid a$
2. $A \rightarrow ad \mid a \mid ab \mid abc \mid x$
3. $S \rightarrow aSSbS \mid aSaSb \mid abb \mid b$
4. $A \rightarrow aAB \mid aBc \mid aAc$
5. $S \rightarrow aAd \mid aB$
 $A \rightarrow a \mid ab$
 $B \rightarrow ccd \mid ddc$

1. $S \rightarrow iEtS \mid iEtSeS \mid a$

Soln:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow \epsilon \mid es$$

2. $A \rightarrow ad \mid a \mid ab \mid abc \mid x$

Soln:

$$A \rightarrow aA' \mid x$$

$$A' \rightarrow d \mid \epsilon \mid b \mid bc$$

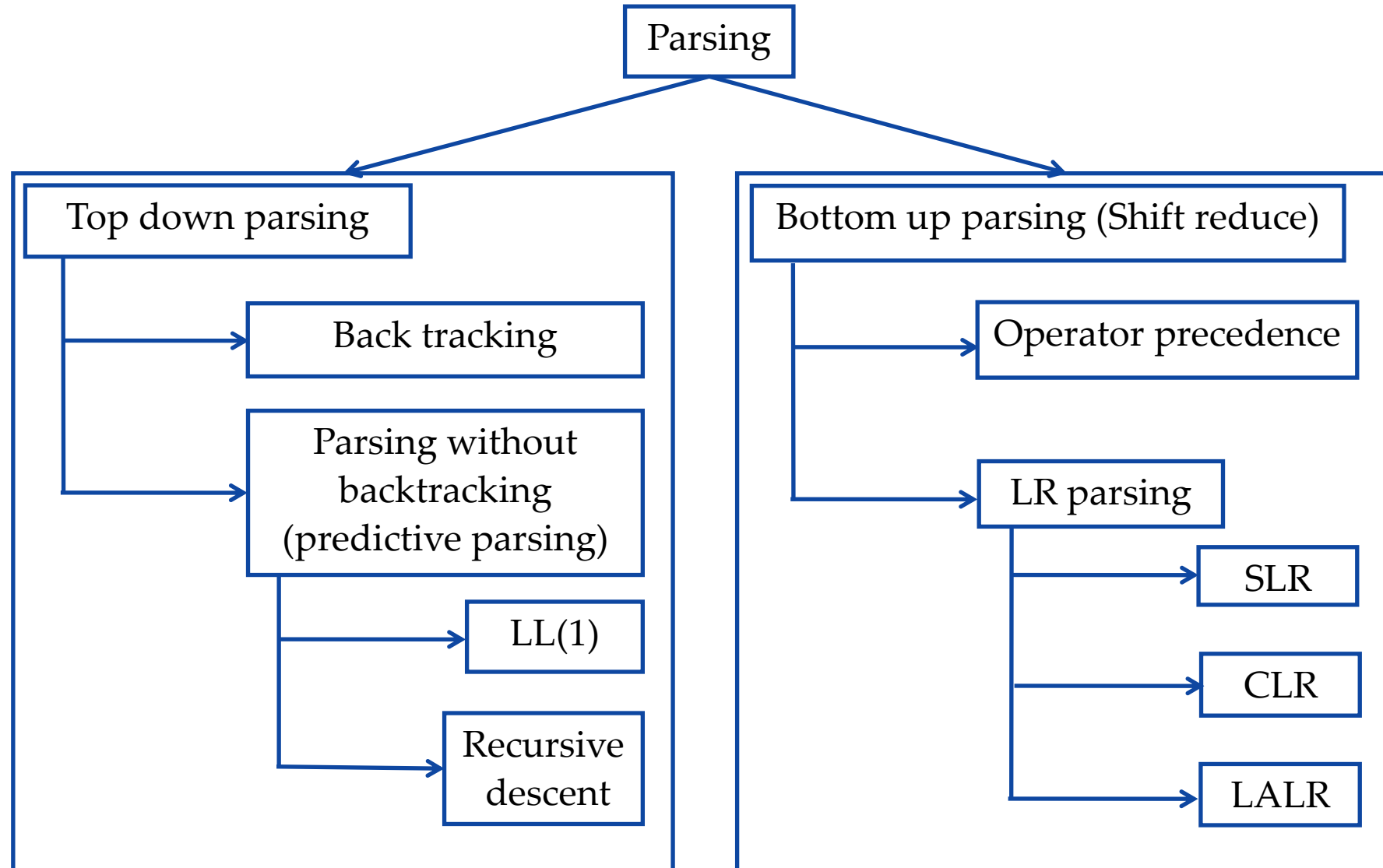
$$A' \rightarrow bA''$$

$$A'' \rightarrow c \mid \epsilon$$

Parsing

- Parsing is a technique that takes input string and produces output either a **parse tree** if string is valid sentence of grammar, or an **error message** indicating that string is not a valid.
- Types of parsing are:
 - 1. Top down parsing:** In top down parsing parser build parse tree from top to bottom.
 - 2. Bottom up parsing:** Bottom up parser starts from leaves and work up to the root.

Classification of parsing methods



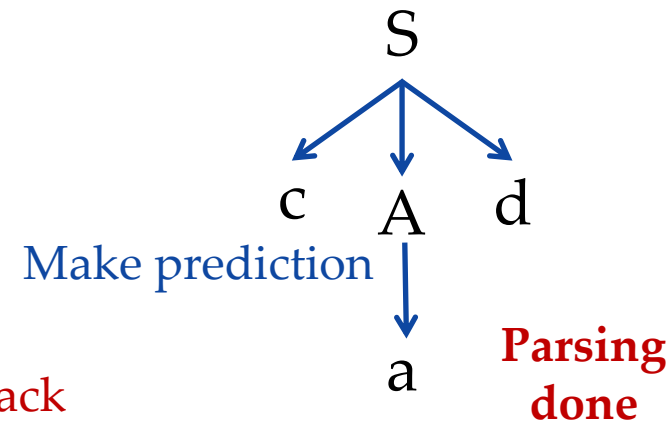
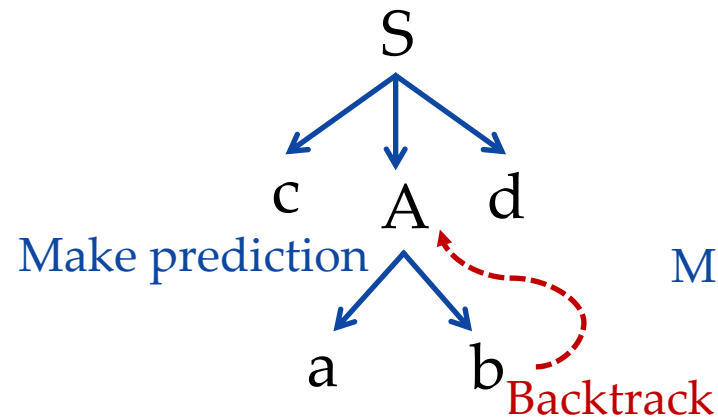
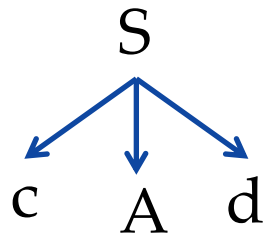
Backtracking

- In backtracking, expansion of nonterminal symbol we choose one alternative and if any mismatch occurs then we try another alternative.

• Grammar: $S \rightarrow cAd$

Input string: cad

$A \rightarrow ab \mid a$



Exercise

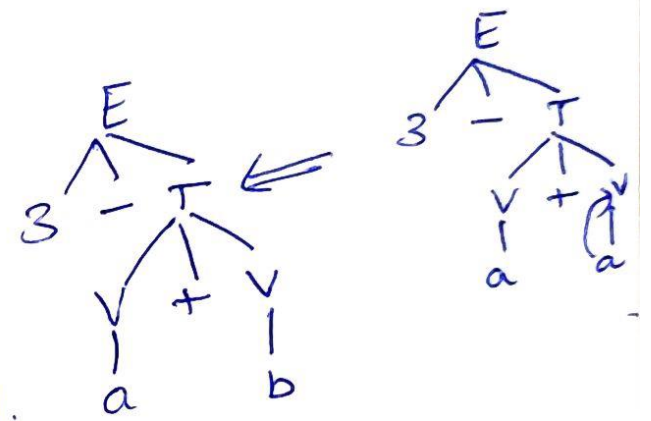
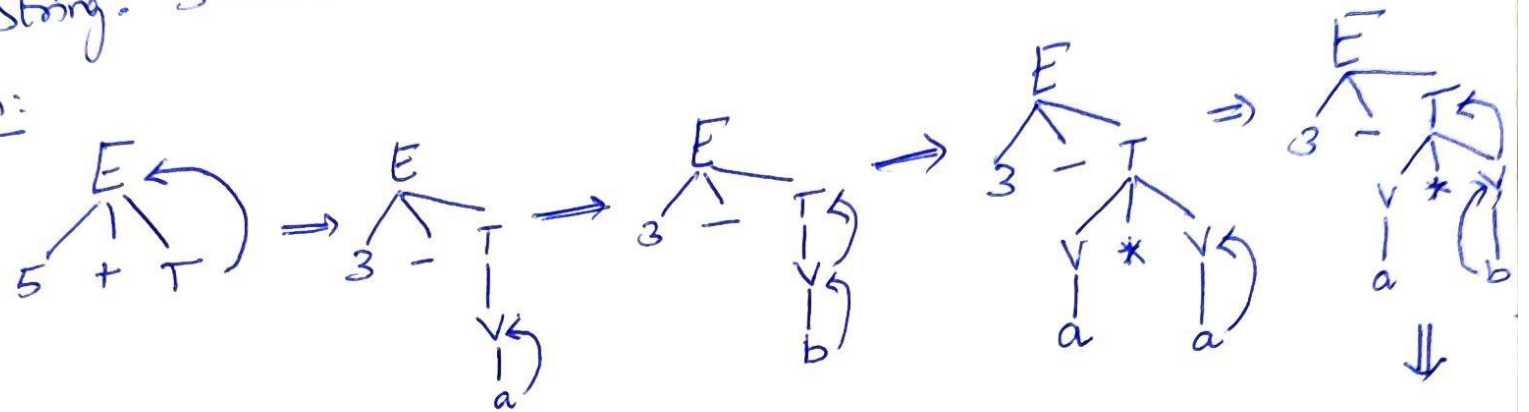
1. $E \rightarrow 5+T \mid 3-T$

$$T \rightarrow V \mid V^*V \mid V_+V$$
$$V \rightarrow a \mid b$$

String: 3-a+b

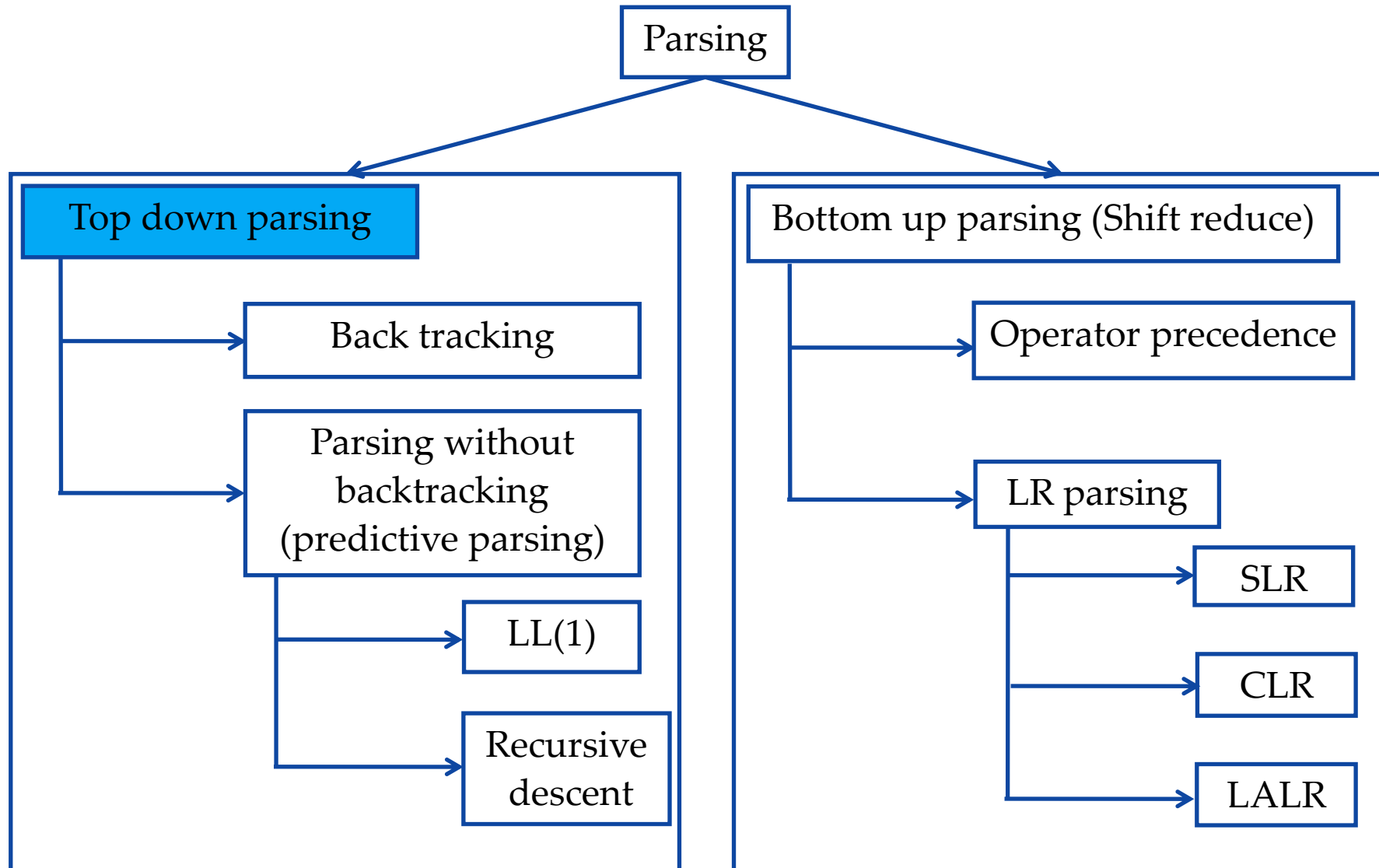
1) $E \rightarrow B + T \mid 3 - T$
 $T \rightarrow V \mid V * V \mid V + V$
 $V \rightarrow a \mid b$
 String: $3 - a + b$

Soln:



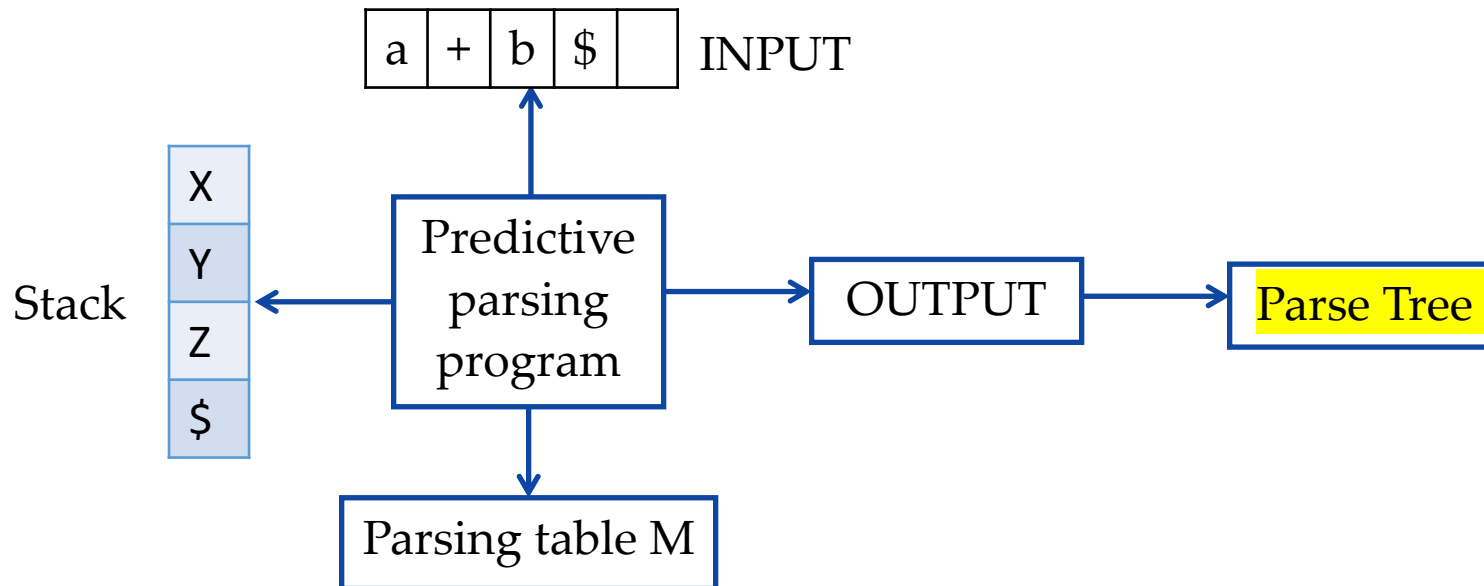
Parsing Completed.

Parsing Methods



LL(1) parser (predictive parser)

- LL(1) is non recursive top down parser.
 1. First **L** indicates *input is scanned from left to right*.
 2. The second **L** means it *uses leftmost derivation for input string*
 3. **1** means it *uses only input symbol to predict the parsing process*.
(how many symbols are you going to see when you want to make a decision)



LL(1) parsing (predictive parsing)

Steps to construct LL(1) parser

1. Remove left recursion / Perform left factoring (if any).
2. Compute FIRST and FOLLOW of non terminals.
3. Construct predictive parsing table.
4. Parse the input string using parsing table.

Rules to compute **first** of non terminal

1. If $A \rightarrow \alpha$ and α is terminal, add α to $FIRST(A)$.
2. If $A \rightarrow \epsilon$, add ϵ to $FIRST(A)$.
3. If X is nonterminal and $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production, then place a in $FIRST(X)$ if for some i , a is in $FIRST(Y_i)$, and ϵ is in all of $FIRST(Y_1), \dots, FIRST(Y_{i-1})$; that is $Y_1 \dots Y_{i-1} \Rightarrow \epsilon$. If ϵ is in $FIRST(Y_j)$ for all $j = 1, 2, \dots, k$ then add ϵ to $FIRST(X)$.

Everything in $FIRST(Y_1)$ is surely in $FIRST(X)$ If Y_1 does not derive ϵ , then we do nothing more to $FIRST(X)$, but if $Y_1 \Rightarrow \epsilon$, then we add $FIRST(Y_2)$ and so on.

Rules to compute **first** of non terminal

Simplification of Rule 3

If $A \rightarrow Y_1 Y_2 \dots \dots Y_K$,

- If Y_1 **does not derives** ϵ then, $FIRST(A) = FIRST(Y_1)$

- If Y_1 **derives** ϵ then,

$$FIRST(A) = FIRST(Y_1) - \epsilon \cup FIRST(Y_2)$$

- If Y_1 & Y_2 **derives** ϵ then,

$$FIRST(A) = FIRST(Y_1) - \epsilon \cup FIRST(Y_2) - \epsilon \cup FIRST(Y_3)$$

- If Y_1 , Y_2 & Y_3 **derives** ϵ then,

$$FIRST(A) = FIRST(Y_1) - \epsilon \cup FIRST(Y_2) - \epsilon \cup FIRST(Y_3) - \epsilon \cup FIRST(Y_4)$$

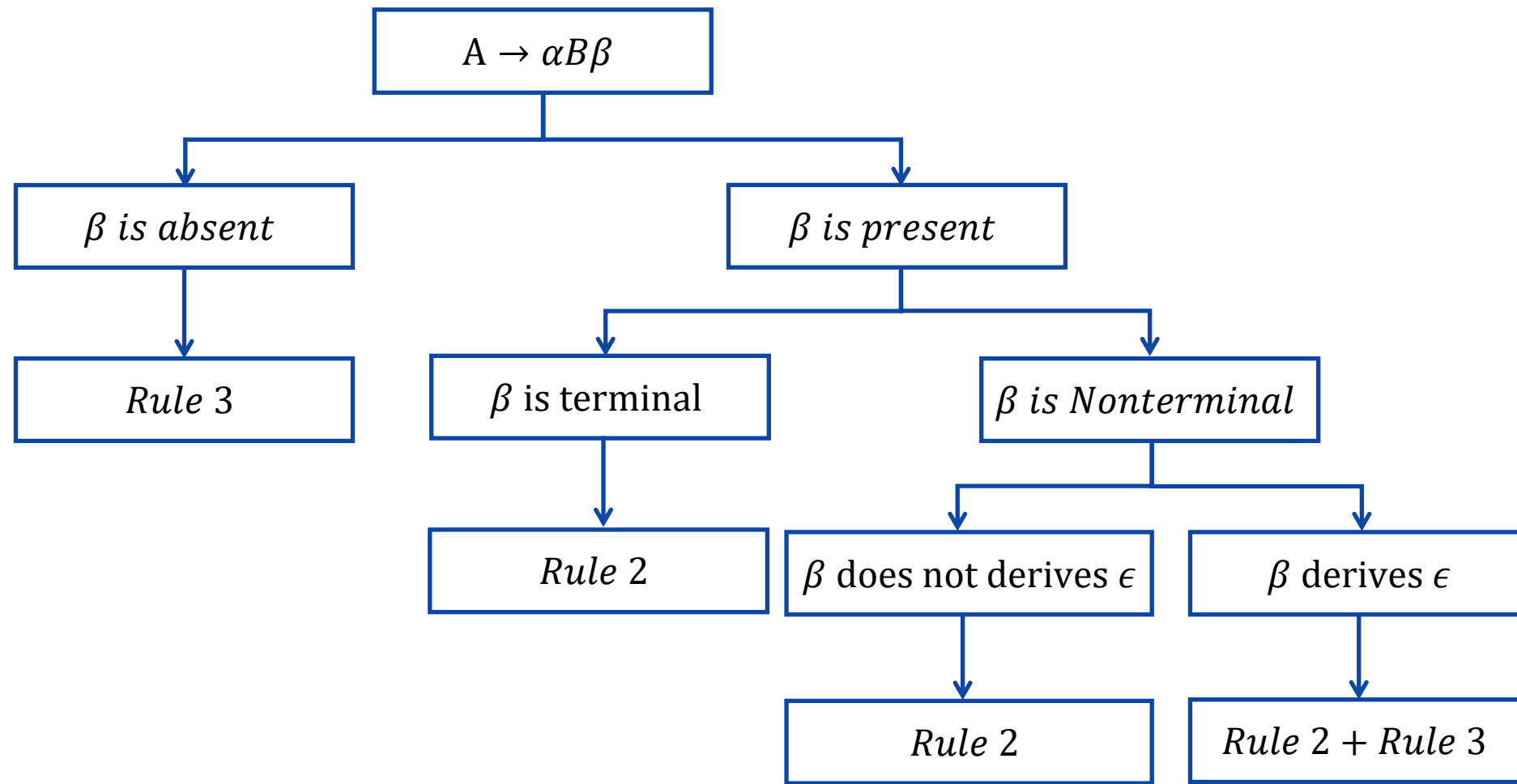
- If Y_1 , Y_2 , $Y_3 \dots Y_K$ all **derives** ϵ then,

$$FIRST(A) = FIRST(Y_1) - \epsilon \cup FIRST(Y_2) - \epsilon \cup FIRST(Y_3) - \epsilon \cup FIRST(Y_4) - \epsilon \cup \dots \dots FIRST(Y_K) \text{ (note: if all non terminals derives } \epsilon \text{ then add } \epsilon \text{ to } FIRST(A))$$

Rules to compute FOLLOW of non terminal

1. Place \$ in $follow(S)$. (S is start symbol)
2. If $A \rightarrow \alpha B \beta$, then everything in $FIRST(\beta)$ except for ϵ is placed in $FOLLOW(B)$
3. If there is a production $A \rightarrow \alpha B$ or a production $A \rightarrow \alpha B \beta$ where $FIRST(\beta)$ contains ϵ then everything in $FOLLOW(A) = FOLLOW(B)$

How to apply rules to find FOLLOW of non terminal?



Rules to construct predictive parsing table

1. For each production $A \rightarrow \alpha$ of the grammar, do steps 2 and 3.
2. For each terminal a in $first(\alpha)$, Add $A \rightarrow \alpha$ to $M[A, a]$.
3. If ϵ is in $first(\alpha)$, Add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal b in $FOLLOW(B)$. If ϵ is in $first(\alpha)$, and $\$$ is in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$.
4. Make each undefined entry of M be error.

Steps to construct LL(1) parsing (predictive parsing)

1. Remove left recursion / Perform left factoring (if any).

2. Compute **FIRST** and **FOLLOW** of non terminals.

a. **FIRST**

i. If $S \rightarrow aSBc$, $B \rightarrow \epsilon$ then **FIRST**(S) = {a}, **FIRST**(B) = { ϵ }

ii. If $A \rightarrow Bxy$, $B \rightarrow MN$, $M \rightarrow z$ then **FIRST**(A) = **FIRST**(B) = **FIRST**(M) = {z}

b. **FOLLOW**

i. Place \$ in *follow*(S). (S is start symbol)

ii. If Terminal found on **FOLLOW**(NT), then add that terminal as **FOLLOW**(NT) = Terminal

iii. If No symbol as **FOLLOW**(NT), then **FOLLOW**(NT) = **FOLLOW**(NT on LHS)

iv. If NT found as **FOLLOW**(NT) then, Find **FIRST**(**FOLLOW**(NT)) then apply ϵ and find **FOLLOW**(NT on LHS)

3. Construct predictive parsing table.

i. For each N.T, if **FIRST**(NT) is not ϵ then for each symbols of **FIRST**(NT_{RHS}) update its production in the parsing table.

ii. If **FIRST**(NT) is ϵ , the for each symbols of **FOLLOW**(NT_{LHS}) update its production in the parsing table.

iii. Make each undefined entry of Parsing Table as error.

4. Parse the input string using parsing table.

Example-1: LL(1) parsing

Construct predictive parsing table for the following grammar:

$S \rightarrow aBa$

$B \rightarrow bB \mid \epsilon$

Step 1: Not required

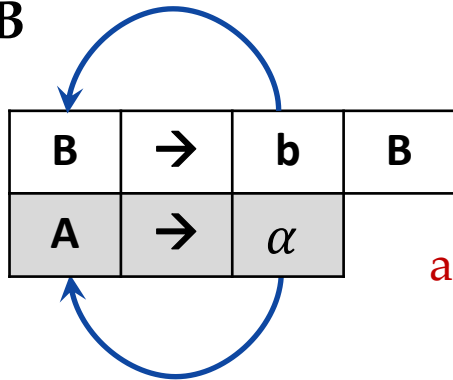
Step 2: Compute FIRST

First(S)

$S \rightarrow aBa$

First(B)

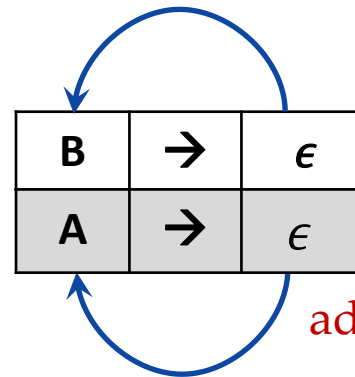
$B \rightarrow bB$



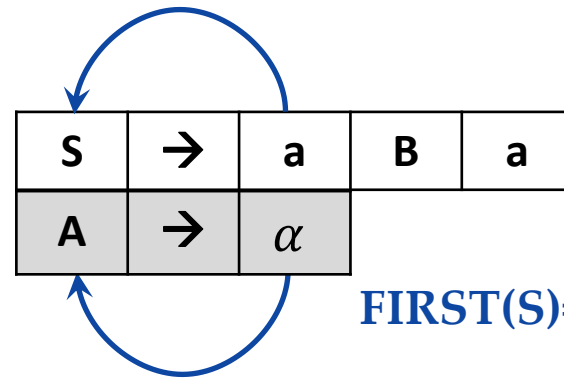
Rule 1
add α to $FIRST(A)$

$FIRST(B) = \{ b, \epsilon \}$

$B \rightarrow \epsilon$



Rule 2
add ϵ to $FIRST(A)$



Rule 1
add α to $FIRST(A)$

$FIRST(S) = \{ a \}$

NT	First
S	
B	

Example-1: LL(1) parsing

$S \rightarrow aBa$

$B \rightarrow bB \mid \epsilon$

Step 3: Compute FOLLOW

Follow(S)

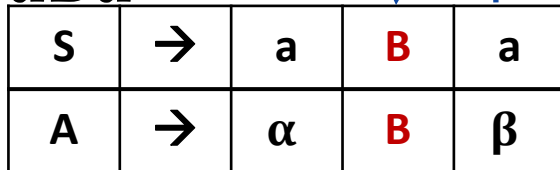
Rule 1: Place \$ in FOLLOW(S)

$\text{Follow}(S) = \{ \$ \}$

NT	First	Follow
S	{a}	
B	{b, ϵ }	

Follow(B)

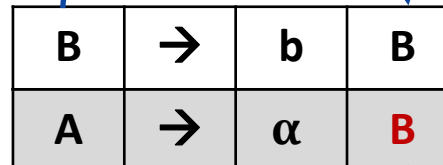
$S \rightarrow aBa$



S	$\rightarrow$	a	B	a
A	$\rightarrow$	α	B	β

Rule 2
 $\text{First}(\beta) = \epsilon$

$B \rightarrow bB$



B	$\rightarrow$	b	B
A	$\rightarrow$	α	B

Rule 3
 $\text{Follow}(A) = \text{follow}(B)$

$\text{Follow}(B) = \{ a \}$

Example-1: LL(1) parsing

$S \rightarrow aBa$

$B \rightarrow bB \mid \epsilon$

Step 4: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{a}

N T	Input Symbol		
	a	b	\$
S			
B			

$S \rightarrow aBa$

$a = \text{FIRST}(aBa) = \{ a \}$

$M[S, a] = S \rightarrow aBa$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-1: LL(1) parsing

$S \rightarrow aBa$

$B \rightarrow bB \mid \epsilon$

Step 4: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{a}

N T	Input Symbol		
	a	b	\$
S	$S \rightarrow aBa$		
B			

$B \rightarrow bB$

$a = \text{FIRST}(bB) = \{ b \}$

$M[B, b] = B \rightarrow bB$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-1: LL(1) parsing

$S \rightarrow aBa$

$B \rightarrow bB \mid \epsilon$

Step 4: Prepare predictive parsing table

NT	First	Follow
S	{a}	{ \$ }
B	{b, ϵ }	{a}

N T	Input Symbol		
	a	b	\$
S	$S \rightarrow aBa$		
B		$B \rightarrow bB$	

$B \rightarrow \epsilon$

$b = \text{FOLLOW}(B) = \{ a \}$

$M[B, a] = B \rightarrow \epsilon$

Rule: 3

$A \rightarrow \alpha$

$b = \text{follow}(A)$

$M[A, b] = A \rightarrow \alpha$

Example-2: LL(1) parsing

Construct predictive parsing table for the following grammar:

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

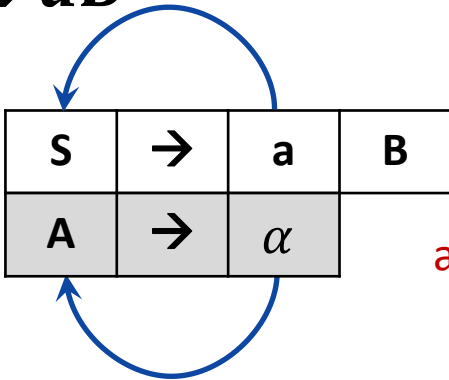
$C \rightarrow cS \mid \epsilon$

Step 1: Not required

Step 2: Compute FIRST

$\text{First}(S)$

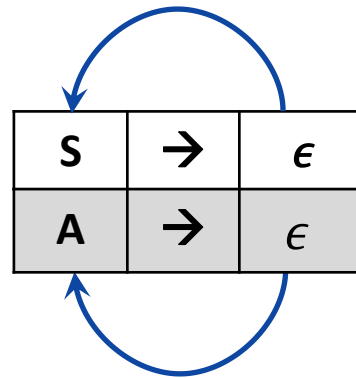
$S \rightarrow aB$



Rule 1
add α to $\text{FIRST}(A)$

$\text{FIRST}(S) = \{ a, \epsilon \}$

$S \rightarrow \epsilon$



Rule 2
add ϵ to $\text{FIRST}(A)$

NT	First
S	
B	
C	

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

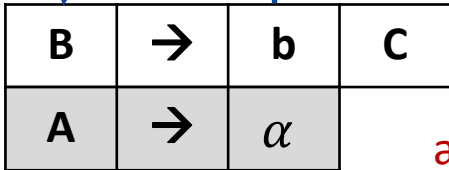
$C \rightarrow cS \mid \epsilon$

Step 1: Not required

Step 2: Compute FIRST

First(B)

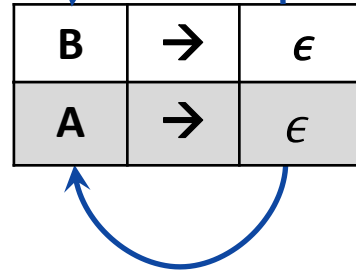
$B \rightarrow bC$



Rule 1
add α to $FIRST(A)$

$FIRST(B) = \{ b, \epsilon \}$

$B \rightarrow \epsilon$



Rule 2
add ϵ to $FIRST(A)$

NT	First
S	{ a, ε }
B	
C	

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

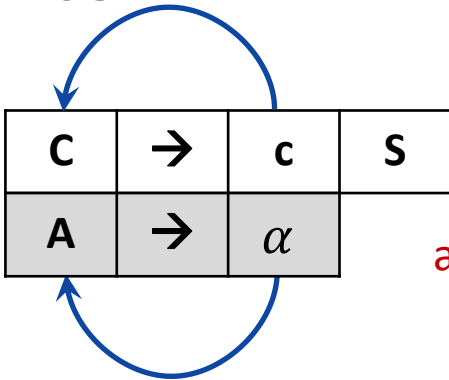
$C \rightarrow cS \mid \epsilon$

Step 1: Not required

Step 2: Compute FIRST

$\text{First}(C)$

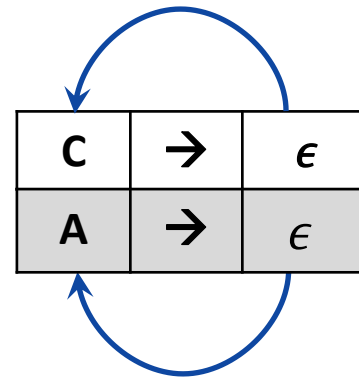
$C \rightarrow cS$



Rule 1
add α to $\text{FIRST}(A)$

$\text{FIRST}(B) = \{c, \epsilon\}$

$C \rightarrow \epsilon$



Rule 2
add ϵ to $\text{FIRST}(A)$

NT	First
S	{a,ε}
B	{b,ε}
C	

Example-2: LL(1) parsing

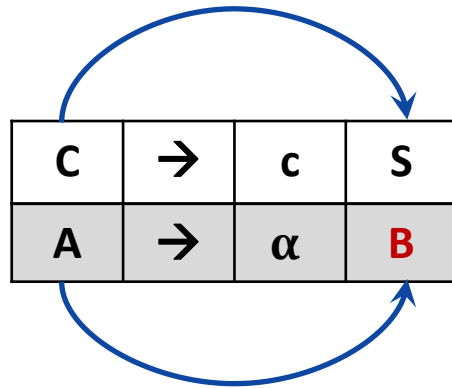
Step 2: Compute FOLLOW

Follow(S)

Rule 1: Place \$ in FOLLOW(S)

Follow(S)={ \$ }

$C \rightarrow cS$

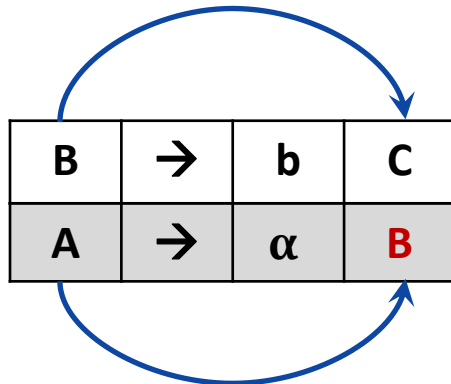


Rule 3

Follow(A)=follow(B)

Follow(S)=Follow(C)={\$}

$B \rightarrow bC$

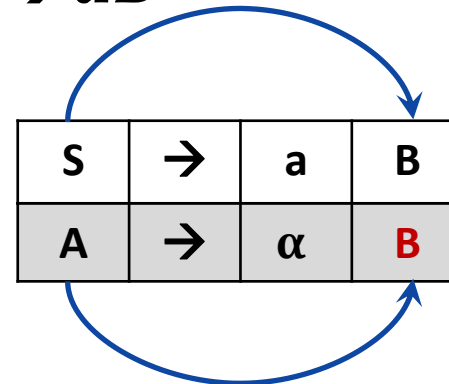


Rule 3

Follow(A)=follow(B)

Follow(C)=Follow(B)={\$}

$S \rightarrow aB$



Rule 3

Follow(A)=follow(B)

Follow(B)=Follow(S)={\$}

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

NT	First	Follow
S	{a,ε}	
B	{b,ε}	
C	{c,ε}	

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

Step 3: Prepare predictive parsing table

NT	First	Follow
S	{a, ϵ }	{ $\$$ }
B	{b, ϵ }	{ $\$$ }
C	{c, ϵ }	{ $\$$ }

N T	Input Symbol			
	a	b	c	$\$$
S				
B				
C				

$S \rightarrow aB$

$a = \text{FIRST}(aB) = \{ a \}$

$M[S, a] = S \rightarrow aB$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

Step 3: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{\$}
C	{c, ϵ }	{\$}

N T	Input Symbol			
	a	b	c	\$
S	$S \rightarrow aB$			
B				
C				

$S \rightarrow \epsilon$

$b = \text{FOLLOW}(S) = \{ \$ \}$

$M[S, \$] = S \rightarrow \epsilon$

Rule: 3

$A \rightarrow \alpha$

$b = \text{follow}(A)$

$M[A, b] = A \rightarrow \alpha$

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

Step 3: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{\$}
C	{c, ϵ }	{\$}

N T	Input Symbol			
	a	b	c	\$
S	$S \rightarrow aB$			$S \rightarrow \epsilon$
B				
C				

$B \rightarrow bC$

$a = \text{FIRST}(bC) = \{ b \}$

$M[B, b] = B \rightarrow bC$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

Step 3: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{\$}
C	{c, ϵ }	{\$}

N T	Input Symbol			
	a	b	c	\$
S	$S \rightarrow aB$			$S \rightarrow \epsilon$
B		$B \rightarrow bC$		
C				

$B \rightarrow \epsilon$

$b = \text{FOLLOW}(B) = \{ \$ \}$

$M[B, \$] = B \rightarrow \epsilon$

Rule: 3

$A \rightarrow \alpha$

$b = \text{follow}(A)$

$M[A, b] = A \rightarrow \alpha$

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

Step 3: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{\$}
C	{c, ϵ }	{\$}

N T	Input Symbol			
	a	b	c	\$
S	$S \rightarrow aB$			$S \rightarrow \epsilon$
B		$B \rightarrow bC$		$B \rightarrow \epsilon$
C				

$C \rightarrow cS$

$a = \text{FIRST}(cS) = \{ c \}$

$M[C, c] = C \rightarrow cS$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-2: LL(1) parsing

$S \rightarrow aB \mid \epsilon$

$B \rightarrow bC \mid \epsilon$

$C \rightarrow cS \mid \epsilon$

Step 3: Prepare predictive parsing table

NT	First	Follow
S	{a}	{\$}
B	{b, ϵ }	{\$}
C	{c, ϵ }	{\$}

N T	Input Symbol			
	a	b	c	\$
S	$S \rightarrow aB$			$S \rightarrow \epsilon$
B		$B \rightarrow bB$		$B \rightarrow \epsilon$
C			$C \rightarrow cS$	

$C \rightarrow \epsilon$

$b = \text{FOLLOW}(C) = \{ \$ \}$

$M[C, \$] = C \rightarrow \epsilon$

Rule: 3

$A \rightarrow \alpha$

$b = \text{follow}(A)$

$M[A, b] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Construct LL(1) parser for the following grammar:

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T*F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

Check the acceptance of the input string $\text{id}+\text{id}*\text{id}\$$

Soln:

Step 1: Remove left recursion

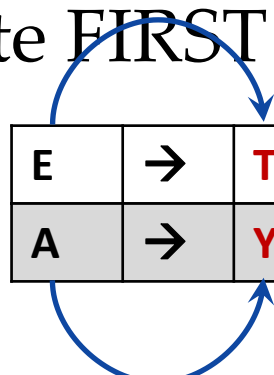
$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \epsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \epsilon$$
$$F \rightarrow (E) \mid \text{id}$$

Example-3: LL(1) parsing

Step 2: Compute FIRST

First(E)

$E \rightarrow TE'$



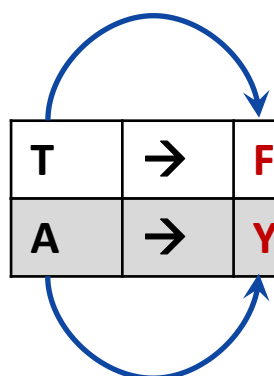
E	$\rightarrow$	T	E'
A	$\rightarrow$	Y ₁	Y ₂

Rule 3
 $\text{First}(A) = \text{First}(Y_1)$

$\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$

First(T)

$T \rightarrow FT'$



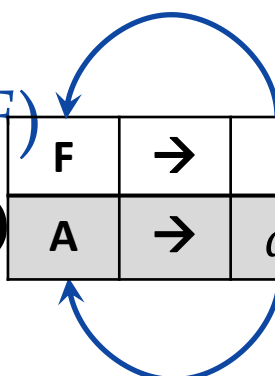
T	$\rightarrow$	F	T'
A	$\rightarrow$	Y ₁	Y ₂

Rule 3
 $\text{First}(A) = \text{First}(Y_1)$

$\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$

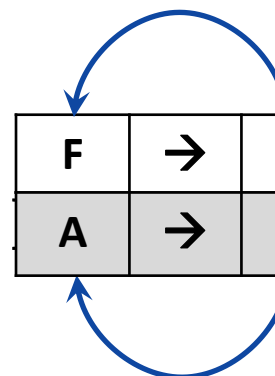
First(F)

$F \rightarrow (E)$



F	$\rightarrow$	(	E	)
A	$\rightarrow$	α		

Rule 1
add α to $\text{FIRST}(A)$
 $\text{FIRST}(F) = \{ (, \text{id} \}$



F	$\rightarrow$	id
A	$\rightarrow$	α

Rule 1
add α to $\text{FIRST}(A)$

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid \text{id}$

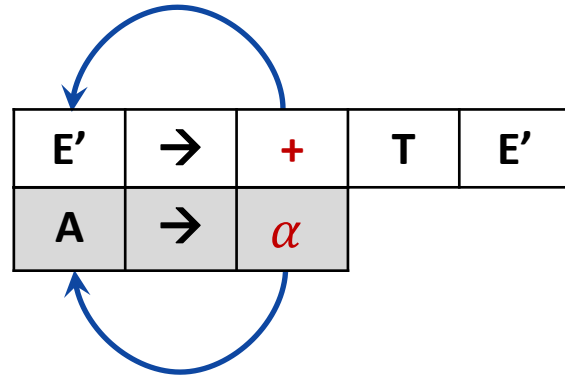
NT	First
E	
E'	
T	
T'	
F	

Example-3: LL(1) parsing

Step 2: Compute FIRST

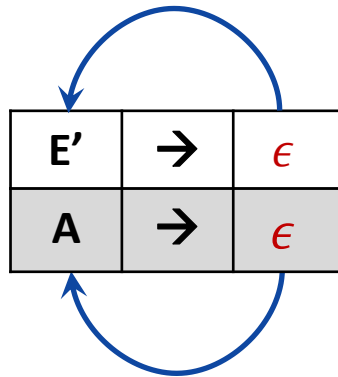
First(E')

$E' \rightarrow +TE'$



Rule 1
add α to $FIRST(A)$

$E' \rightarrow \epsilon$



Rule 2
add ϵ to $FIRST(A)$

$FIRST(E') = \{ +, \epsilon \}$

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

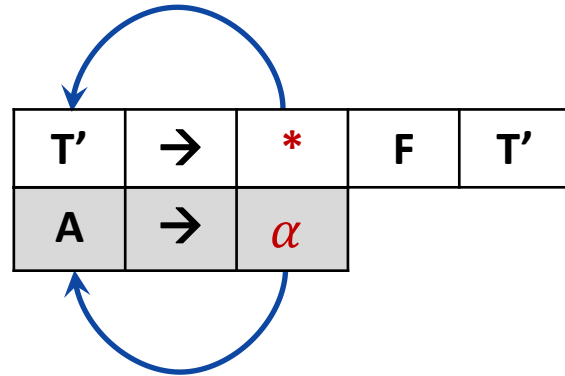
NT	First
E	{ (, id }
E'	
T	{ (, id }
T'	
F	{ (, id }

Example-3: LL(1) parsing

Step 2: Compute FIRST

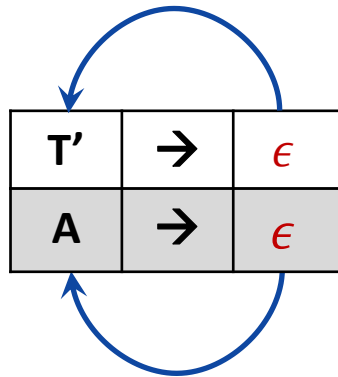
First(T')

$T' \rightarrow *FT'$



Rule 1
add α to $FIRST(A)$

$T' \rightarrow \epsilon$



Rule 2
add ϵ to $FIRST(A)$

$FIRST(T') = \{ *, \epsilon \}$

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First
E	{ (, id }
E'	{ +, ϵ }
T	{ (, id }
T'	
F	{ (, id }

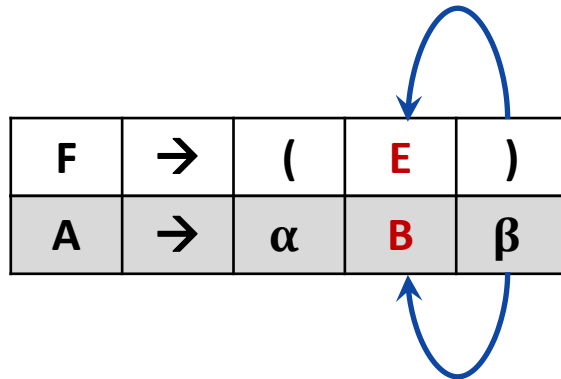
Example-3: LL(1) parsing

Step 2: Compute FOLLOW

FOLLOW(E)

Rule 1: Place \$ in FOLLOW(E)

$F \rightarrow (E)$



Rule 2

FOLLOW(E) = { \$,) }

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

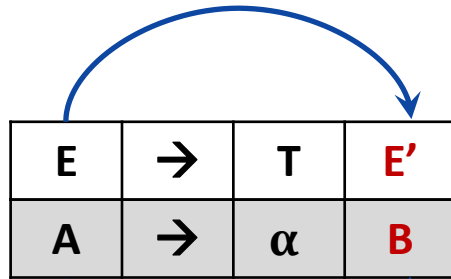
NT	First	Follow
E	{ (, id }	
E'	{ +, ε }	
T	{ (, id }	
T'	{ *, ε }	
F	{ (, id }	

Example-3: LL(1) parsing

Step 2: Compute FOLLOW

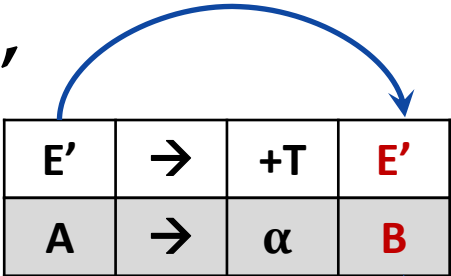
$\text{FOLLOW}(E')$

$E \rightarrow TE'$



Rule 3

$E' \rightarrow +TE'$



Rule 3

$\text{FOLLOW}(E') = \{ \$,) \}$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	
T	{ (, id }	
T'	{ *, ϵ }	
F	{ (, id }	

Example-3: LL(1) parsing

Step 2: Compute FOLLOW

FOLLOW(T)

$E \rightarrow TE'$

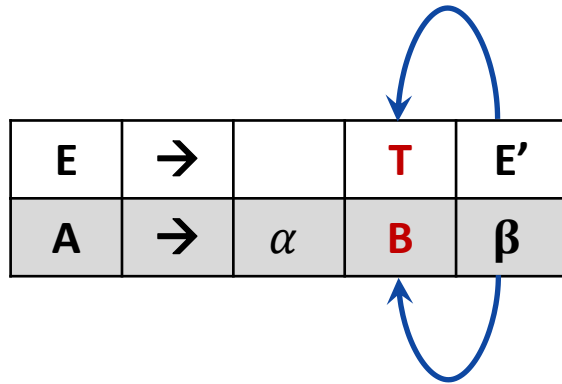
$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

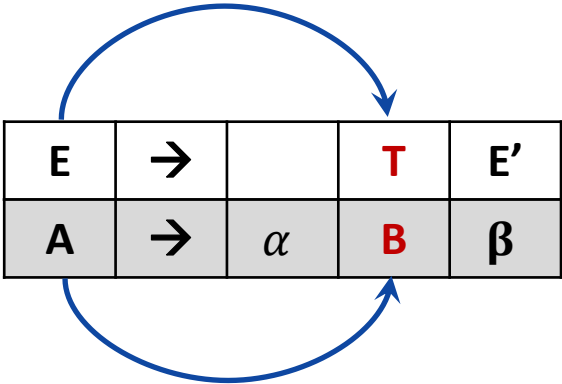
$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$



Rule 2



Rule 3

$\text{FOLLOW}(T) = \{ +, \$,) \}$

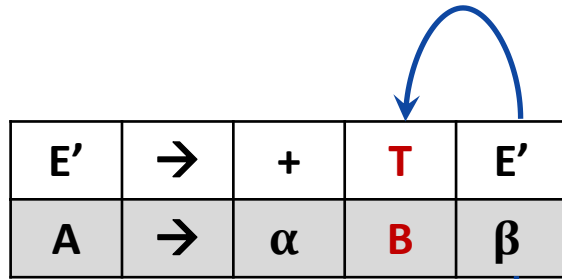
NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	
T'	{ *, ϵ }	
F	{ (, id }	

Example-3: LL(1) parsing

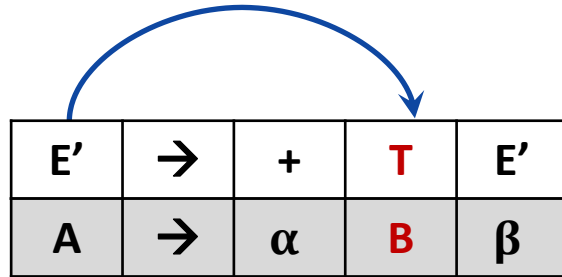
Step 2: Compute FOLLOW

FOLLOW(T)

$E' \rightarrow +TE'$



Rule 2



Rule 3

$\text{FOLLOW}(T) = \{ +, \$,) \}$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

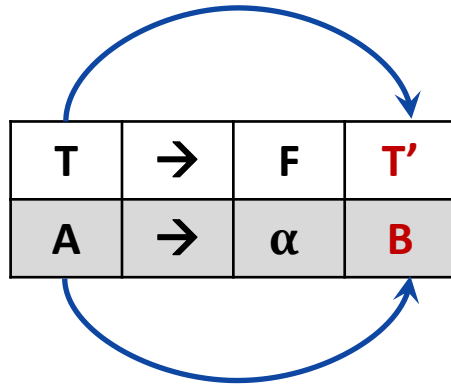
NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	
T'	{ *, ϵ }	
F	{ (, id }	

Example-3: LL(1) parsing

Step 2: Compute FOLLOW

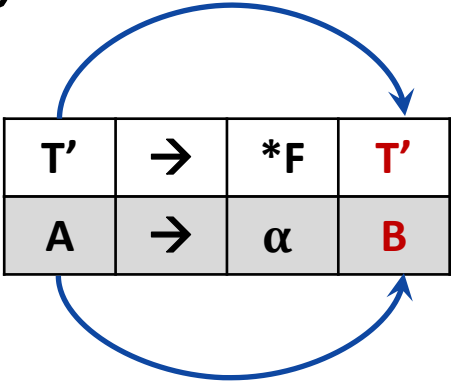
FOLLOW(T')

$T \rightarrow FT'$



Rule 3

$T' \rightarrow *FT'$



Rule 3

FOLLOW(T') = {+, \$,)}

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	
F	{ (, id }	

Example-3: LL(1) parsing

Step 2: Compute FOLLOW

FOLLOW(F)

$T \rightarrow FT'$

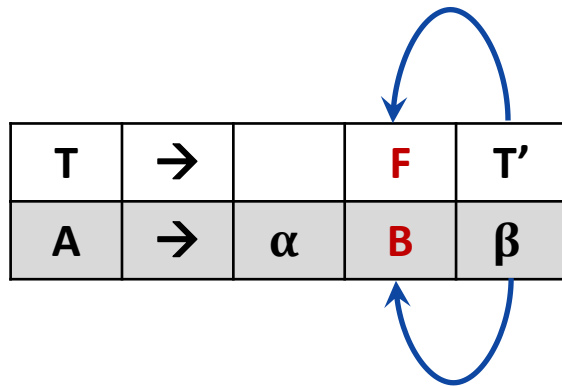
$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

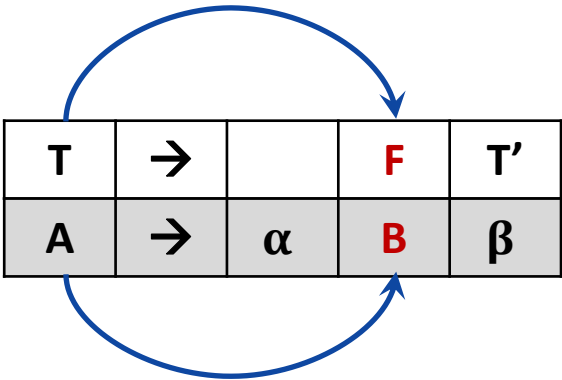
$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$



Rule 2



Rule 3

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ε }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ε }	{ +, \$,) }
F	{ (, id }	

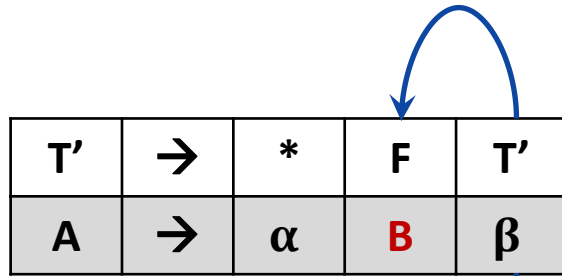
$FOLLOW(F) = \{ *, +, \$,) \}$

Example-3: LL(1) parsing

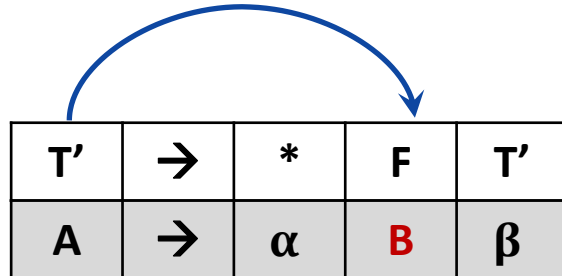
Step 2: Compute FOLLOW

FOLLOW(F)

$T' \rightarrow *FT'$



Rule 2



Rule 3

$\text{FOLLOW}(F) = \{ *, +, \$,) \}$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E						
E'						
T						
T'						
F						

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

$E \rightarrow TE'$

$a = \text{FIRST}(TE') = \{ (, id \}$

$M[E, (] = E \rightarrow TE'$

$M[E, id] = E \rightarrow TE'$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'						
T						
T'						
F						

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

$E' \rightarrow +TE'$

$a = \text{FIRST}(+TE') = \{ + \}$

$M[E', +] = E' \rightarrow +TE'$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$				
T						
T'						
F						

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

$E' \rightarrow \epsilon$

$b = \text{FOLLOW}(E') = \{ \$,) \}$

$M[E', \$] = E' \rightarrow \epsilon$

$M[E',)] = E' \rightarrow \epsilon$

Rule: 3

$A \rightarrow \alpha$

$b = \text{follow}(A)$

$M[A, b] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T						
T'						
F						

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

$T \rightarrow FT'$

$a = \text{FIRST}(FT') = \{ (, id \}$

$M[T, (] = T \rightarrow FT'$

$M[T, id] = T \rightarrow FT'$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'						
F						

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

$T' \rightarrow *FT'$

$a = \text{FIRST}(*FT') = \{ * \}$

$M[T', *] = T' \rightarrow *FT'$

Rule: 2
 $A \rightarrow \alpha$
 $a = \text{first}(\alpha)$
 $M[A, a] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'			$T' \rightarrow *FT'$			
F						

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

$T' \rightarrow \epsilon$

$b = \text{FOLLOW}(T') = \{ +, \$,) \}$

$M[T', +] = T' \rightarrow \epsilon$

$M[T', \$] = T' \rightarrow \epsilon$

$M[T',)] = T' \rightarrow \epsilon$

Rule: 3

$A \rightarrow \alpha$

$b = \text{follow}(A)$

$M[A, b] = A \rightarrow \alpha$

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F						

$F \rightarrow (E)$

$a = \text{FIRST}((E)) = \{ (\}$

$M[F, (] = F \rightarrow (E)$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

Example-3: LL(1) parsing

Step 3: Construct predictive parsing table

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F				$F \rightarrow (E)$		

$F \rightarrow id$

$a = \text{FIRST}(id) = \{ id \}$

$M[F, id] = F \rightarrow id$

Rule: 2

$A \rightarrow \alpha$

$a = \text{first}(\alpha)$

$M[A, a] = A \rightarrow \alpha$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

NT	First	Follow
E	{ (, id }	{ \$,) }
E'	{ +, ϵ }	{ \$,) }
T	{ (, id }	{ +, \$,) }
T'	{ *, ϵ }	{ +, \$,) }
F	{ (, id }	{ *, +, \$,) }

Example-3: LL(1) parsing

- Step 4: Make each undefined entry of table be Error

NT	Input Symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$	Error	Error	$E \rightarrow TE'$	Error	Error
E'	Error	$E' \rightarrow +TE'$	Error	Error	$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	Error	Error	$T \rightarrow FT'$	Error	Error
T'	Error	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	Error	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$	Error	Error	$F \rightarrow (E)$	Error	Error

Exercise

1. Construct parsing table for the following grammar:

$S \rightarrow aABb$

$A \rightarrow c \mid \epsilon$

$B \rightarrow d \mid \epsilon$

2. Construct parsing table for the following grammar:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

3. Construct parsing table for the following grammar:

$S \rightarrow AB \mid eDa$

$A \rightarrow ab \mid c$

$B \rightarrow dC$

$C \rightarrow eC \mid \epsilon$

$D \rightarrow fD \mid \epsilon$

Solution to Ex: 1

$S \rightarrow aABb \quad A \rightarrow c \mid \epsilon \quad B \rightarrow d \mid \epsilon$

Step: 1: No left recursion in the grammar, hence no modification required.

Step 2: Calculation of First Set

$\text{First}(S) = \{a\}$

$\text{First}(A) = \{c, \epsilon\}$

$\text{First}(B) = \{d, \epsilon\}$

Step 3: Calculation of Follow Set

$\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A) = \text{First}(Bb) = \text{First}(B) = \{d, \epsilon\}$

Since it contains ϵ , continue FIRST rule. $\text{First}(Bb) = \text{First}(B) - \epsilon \cup \text{First}(b) = \{d, b\}$

$\text{Follow}(A) = \{d, b\}$

$\text{Follow}(B) = \{b\}$

Step 4: Parsing Table

Grammar is LL(1)

Example String: acdb\$ H/w string: adb\$

	a	b	c	d	\$
S	S→aABb				
A		A→ε	A→c	A→ε	
B		B→ε		B→d	

Example String: acdb\$

	a	b	c	d	\$
S	S->aABb				
A		A->€	A->c	A->€	
B		B->€		B->d	

Stack	Input	Action
\$	acdb\$	Push S into Stack
\$S	acdb\$	S->aABb
\$bBAa	acdb\$	Pop a
\$bBA	cdb\$	A->c
\$bBc	cdb\$	Pop c
\$bB	db\$	B->d
\$bd	db\$	Pop d
\$b	b\$	Pop b
\$	\$	Accept

Solution to Ex: 2

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Step: 1: No left recursion in the grammar, hence no modification required.

Step 2: Calculation of First Set

$\text{First}(S) = \text{First}(AaAb) \cup \text{First}(BbBa)$ $\text{First}(AaAb) =$

$\text{First}(A) = \epsilon$

Since it contains ϵ , continue FIRST rule

$\text{First}(AaAb) = \text{First}(A) - \epsilon \cup \text{First}(aAb) = \{a\}$ Similarly:

$\text{First}(BbBa) = \{b\}$

$\text{First}(S) = \{a, b\}$

Step 3: Calculation of Follow Set $\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A) = \text{First}(aAb) = a$

$\text{Follow}(A) = \{a, b\}$

$\text{Follow}(A) = \text{First}(b) = \{b\}$

Similarly,

$\text{Follow}(B) = \{a, b\}$

Step 4: Construction of Parsing Table: Grammar is LL(1)

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

Solution to Ex: 3

$S \rightarrow AB \mid eDa$

$A \rightarrow ab \mid c$

$B \rightarrow dC$

$C \rightarrow eC \mid \epsilon$

$D \rightarrow fD \mid \epsilon$

Step: 1: No left recursion in the grammar, hence no modification required.

Step 2: Calculation of First Set

First(S) = {a,c,e}

First(A) = {a,c}

First(B) = {d}

First(C) = {e, ϵ }

First(D) = {f, ϵ }

Step 3: Calculation of Follow Set

$\text{Follow}(S) = \{\$ \}$

$S \rightarrow AB$

$\text{Follow}(A) = \text{First}(B) = \{d\}$ $\text{Follow}(B) = \text{Follow}(S) = \{\$ \}$

$S \rightarrow eDA$

$\text{Follow}(D) = \text{First}(a) = \{a\}$

$B \rightarrow dC$

$\text{Follow}(C) = \text{Follow}(B) = \{\$ \}$

$C \rightarrow eC$

$\text{Follow}(C) = \text{Follow}(C) = \{\$ \}$

$D \rightarrow fD$

$\text{Follow}(D) = \text{Follow}(D) = \{a\}$

Parsing Table

	a	b	c	d	e	f	\$
S	S->AB		S->AB		S->eDa		
A	A->ab		A->c				
B		B->dC					
C					C->eC		C->€
D	D->€					D->fD	